

Northeastern University

Project Report – Object Classification

(Milestone 1)

IE 7615 - Discriminative Deep Learning
Spring 2026
February 1, 2026

Group 3

Rithika Sankar Rajeswari	002311800
Meena Periasamy	002311852
Sreevarshan Sathiyamurthy	002311854
Sahil Mohanty	002317489

Repository

GitHub Project Repository: <https://github.com/rithika-sr/discriminative-deep-learning-project>

All source code, training scripts, and notebooks are available in the repository. This report focuses on **experimental setup, results, analysis, and conclusions**.

1. Introduction

This project investigates deep learning approaches for single-object image classification using convolutional neural networks (CNNs). The primary objective of Milestone 1 is to evaluate and compare multiple CNN architectures on a student-collected dataset, establishing a strong classification baseline for future extension into multi-object detection and localization in Milestone 2. Both custom-designed networks and transfer-learning-based architectures are explored to analyze performance trade-offs in accuracy, convergence speed, and computational efficiency.

2. Dataset Description

The dataset consists of images collected collaboratively by students, where each student contributed images of one unique object. The resulting dataset represents a diverse collection of real-world objects captured under varying lighting conditions, viewpoints, and backgrounds.

Dataset Summary

- **Total object classes:** 35
- **Total images:** 3,708
- **Average images per class:** ~106 (range: 99–144)
- **Image resolution:** 224 × 224 (RGB)
- **Class labels:** OBJ001 – OBJ787

Dataset Split

- **Training set:** 2,963 images
- **Validation set:** 366 images
- **Test set:** 379 images
- **Total:** 3,708 images

Data Inspection Output (35 Classes):

```
=====
DATASET INSPECTION REPORT
=====

1. TOTAL CLASSES: 35
   Class folders: images_OBJ001, images_OBJ002, images_OBJ003, images_OBJ004, images_OBJ005..
   (showing first 5)

2. PER-CLASS ANALYSIS:
=====
OBJ001: 100 images
OBJ002: 100 images
OBJ003: 100 images
OBJ004: 100 images
OBJ005: 100 images
OBJ006: 100 images
OBJ007: 100 images
OBJ008: 100 images
OBJ009: 100 images
OBJ010: 100 images
OBJ012: 110 images
OBJ016: 99 images
OBJ018: 120 images
OBJ019: 100 images
OBJ021: 100 images
OBJ022: 99 images
OBJ027: 100 images
OBJ028: 100 images
OBJ029: 100 images
OBJ031: 100 images
OBJ041: 100 images
OBJ090: 100 images
OBJ095: 101 images
OBJ107: 144 images
OBJ108: 100 images
OBJ111: 100 images
OBJ159: 127 images
OBJ288: 100 images
OBJ222: 108 images
OBJ229: 126 images
OBJ230: 140 images
OBJ300: 100 images
OBJ311: 124 images
OBJ405: 100 images
OBJ787: 100 images
=====
```

Dataset Split Statistics:

```
=====
3. DATASET STATISTICS:
=====
Total images: 3708
Average images per class: 105.9
Min images in a class: 99
Max images in a class: 144
Standard deviation: 12.5

4. IMAGE DIMENSIONS:
=====
224x224: 35 images

5. IMAGE FORMATS:
=====
JPEG: 35 images

6. SIZE VERIFICATION:
=====
Images with correct size (224x224): 35/35
All images are correctly sized!

=====
INSPECTION COMPLETE
=====
```

3. Data Preparation

Train–Validation–Test Split

A **stratified split** was applied to preserve class balance across all subsets:

- Training: **80%**
- Validation: **10%**
- Test: **10%**

Data Augmentation (Training Only)

To improve generalization and mitigate overfitting, the following augmentations were applied:

- Random rotation: $\pm 20^\circ$
- Horizontal flipping
- Width and height shifts: $\pm 20\%$
- Zoom: $\pm 20\%$
- Pixel normalization to $[0, 1]$

4. Model Architectures & Training Setup

4.1 Custom CNN (Baseline)

- Trained from scratch
- Three convolutional blocks ($32 \rightarrow 64 \rightarrow 128$ filters)
- Batch normalization and dropout regularization
- Global average pooling
- Fully connected layer (256 units) $\rightarrow$ 35-class softmax output

4.2 ResNet50 (Transfer Learning)

- Pretrained on ImageNet
- Base layers frozen
- Custom classification head added

4.3 MobileNetV2 (Transfer Learning)

- Lightweight and parameter-efficient architecture
- Pretrained on ImageNet
- Base layers frozen
- Custom classification head added

4.4 EfficientNetB0 (Transfer Learning)

- Compound scaling-based architecture
- Pretrained on ImageNet
- Base layers frozen
- Custom classification head added

Training Configuration

- **Input size:** 224×224 RGB
- **Batch size:** 32
- **Optimizer:** Adam (learning rate = 0.001)
- **Loss function:** Categorical Cross-Entropy
- **Epochs:** Up to 30
- **Early stopping:** Patience = 5
- **Learning rate scheduler:** ReduceLROnPlateau

5. Results & Performance Evaluation

5.1 Quantitative Results

Model	Params	Val Acc	Test Acc	Time (min)	Notes
MobileNetV2	~3.5M	99.18%	98.94%	4.9	Best balance of accuracy & efficiency
Custom CNN	~1.2M	92.90%	92.61%	73.4	Good baseline, trained from scratch
ResNet50	~25M	65.57%	63.06%	40.1	Underperformed, high variance
EfficientNetB0	~5.3M	3.55%	3.69%	4.2	Failed to converge

5.2 Training Curves

Training and validation accuracy/loss curves for all models are shown below.

- CustomCNN:

```
Training completed in 4405.53 seconds

Evaluating CustomCNN...
12/12 ----- 5s 406ms/step
Test Accuracy: 0.9261
Test Loss: 0.2781
Model saved: CustomCNN.keras
Downloading data from https://storage.googleapis.com/tensorflow/tf2keras-checkpoints/tf2keras/tf2keras_v1.0_weights_tf_dim_ordering_tf_kernels_notop.h5
94765736/94765736 ----- 4s 0us/step
```

- MobileNetV2:

```
=====
TRAINING: MobileNetV2
=====
Epoch 1/30
93/93 ----- 27s 258ms/step - accuracy: 0.7435 - loss: 1.0642 - val_accuracy: 0.9563 - val_loss: 0.2096 - learning_rate: 0.0010
Epoch 2/30
93/93 ----- 22s 234ms/step - accuracy: 0.9531 - loss: 0.1982 - val_accuracy: 0.9836 - val_loss: 0.0806 - learning_rate: 0.0010
Epoch 3/30
93/93 ----- 22s 233ms/step - accuracy: 0.9706 - loss: 0.1150 - val_accuracy: 0.9891 - val_loss: 0.0483 - learning_rate: 0.0010
Epoch 4/30
93/93 ----- 22s 237ms/step - accuracy: 0.9818 - loss: 0.0823 - val_accuracy: 0.9891 - val_loss: 0.0524 - learning_rate: 0.0010
Epoch 5/30
93/93 ----- 22s 236ms/step - accuracy: 0.9875 - loss: 0.0586 - val_accuracy: 0.9863 - val_loss: 0.0472 - learning_rate: 0.0010
Epoch 6/30
```

- ResNet50:

```
=====
TRAINING: ResNet50
=====
Epoch 1/30
93/93 ----- 81s 826ms/step - accuracy: 0.1630 - loss: 3.0574 - val_loss: 3.6715 - learning_rate: 0.0010
Epoch 2/30
93/93 ----- 74s 801ms/step - accuracy: 0.2511 - loss: 3.0738 - val_loss: 3.5910 - learning_rate: 0.0010
Epoch 3/30
93/93 ----- 75s 802ms/step - accuracy: 0.3196 - loss: 3.1311 - val_loss: 3.2432 - learning_rate: 0.0010
Epoch 4/30
93/93 ----- 74s 799ms/step - accuracy: 0.3274 - loss: 3.0847 - val_loss: 3.8694 - learning_rate: 0.0010
Epoch 5/30
93/93 ----- 75s 803ms/step - accuracy: 0.3685 - loss: 3.1093 - val_loss: 4.1701 - learning_rate: 0.0010
```

- EfficientNetB0:

```
Training completed in 251.81 seconds

Evaluating EfficientNetB0...
12/12 ----- 7s 467ms/step
/Users/rithika/Desktop/Discriminative_Project/venv/lib/python3.13/site-packages/sklearn/metrics/classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/Users/rithika/Desktop/Discriminative_Project/venv/lib/python3.13/site-packages/sklearn/metrics/classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
Test Accuracy: 0.0369
```

6. Analysis

- Custom CNN:** Initial overfitting was observed but effectively mitigated using data augmentation and dropout. The model achieved a solid baseline accuracy of 92.61%.
- ResNet50:** Despite high capacity, the frozen backbone limited adaptation to the dataset. Fine-tuning deeper layers may improve performance.
- EfficientNetB0:** Failed to converge and collapsed to predicting a single class, suggesting incompatibility between pretrained weights and dataset characteristics.
- MobileNetV2:** Achieved the best performance with minimal parameters, demonstrating excellent generalization and fast convergence.

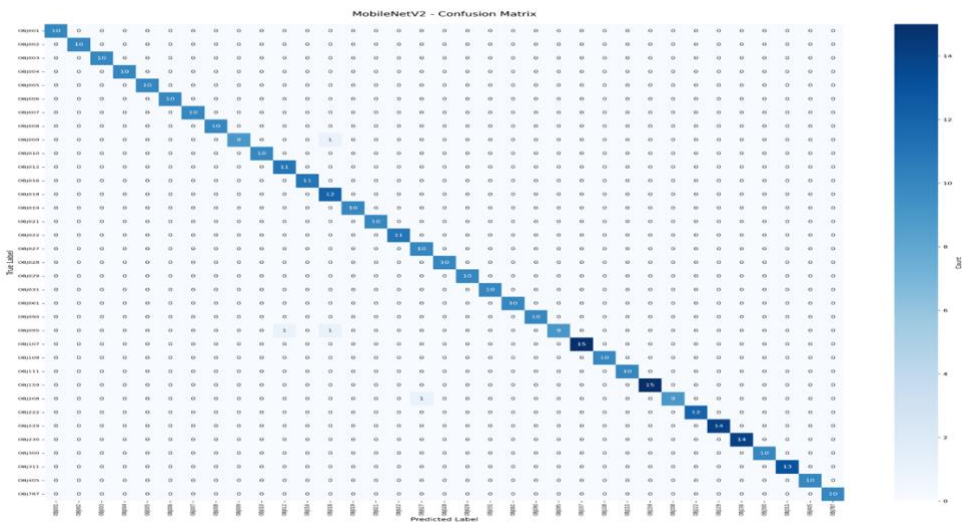
7. Best Model Selection – MobileNetV2

MobileNetV2 was selected as the optimal model due to:

- **Highest test accuracy:** 98.94% (only 4 misclassifications)
- **Fast convergence:** Early stopping at ~13 epochs
- **Computational efficiency:** Only ~3.5M parameters
- **Robust generalization:** Validation accuracy closely matched test accuracy
- **Deployment readiness:** Lightweight architecture suitable for real-world use

7.1 Confusion Matrix

Confusion matrices reveal class-level performance and misclassification patterns.



8. Conclusion

This milestone successfully established a high-performance **single-object image classification pipeline** using convolutional neural networks. Among all evaluated architectures, **MobileNetV2** demonstrated superior accuracy, efficiency, and robustness.

Key Takeaways

- Transfer learning significantly outperformed training from scratch
- Lightweight architectures can outperform deeper models on limited datasets
- Data augmentation played a critical role in generalization
- MobileNetV2 provides a strong foundation for downstream detection tasks

9. Next Steps (Milestone 2)

- Generate synthetic multi-object composite images
- Implement YOLO-based object detection
- Integrate MobileNetV2 for object classification
- Evaluate end-to-end detection + classification pipeline